

Esercizi hazard

1. add **\$t2**, \$t1, \$t0
add \$t5, \$t4, \$t3
add \$t8, \$t7, \$t6
add \$t0, **\$t2**, \$t1

Vi sono possibilità di hazard sulle seguenti istruzioni? se sì quali?

Le prime tre istruzioni non presentano problemi in quanto usano registri diversi. L'ultima istruzione invece presenta un problema in quanto utilizza \$t2 come ingresso, che viene scritto dall'istruzione 1.

Il problema sta nel fatto che quando l'istruzione 4 viene eseguita, il registro \$t2 non è ancora stato aggiornato, cosa che verrà fatta al tempo successivo, terminato lo stadio di write register.

Si supponga di volerle superare inserendo delle nop, quante sono necessarie e dove?

Siccome abbiamo visto che il registro in questione viene scritto al tempo successivo, sarà sufficiente una sola nop, per ritardare l'esecuzione dell'istruzione 4 di uno stadio. Per ottenere questo risultato è sufficiente posizionare la nop prima dell'istruzione 4 o dopo la prima.

L'hardware forwarding può risolvere queste alee?

In questo caso, hardware forwarding risolve completamente il problema. Essendo tutte operazioni aritmetiche che interessano quindi lo stadio di esecuzione, generano una sequenza in cui anche se una operazione dipende dall'istruzione immediatamente precedente, può essere eseguita perchè il risultato della precedente, che serve al tempo attuale per il calcolo dell'istruzione corrente, è già stato calcolato all'istante di tempo precedente.

Esercizio 2

```
beq $t1, $t2, endloop  
i1  
i2  
i3  
endloop:  
jr $ra
```

Potrebbe esserci, non in questo caso, un hazard se nell'istruzione di beq ci fosse l'aggiornamento di un registro che a valle del salto, ovvero nell'istruzione jr \$ra, questo registro deve essere consultato in forma corretta.

Esercizio 3

```
loop:  
beq $t1, $t2, endloop  
i1  
i2  
i3  
addi $t1, $t2, 1  
endloop:  
j loop
```

L'istruzione "addi \$t1, \$t2, 1" si trova 2 istruzioni prima del beq, quindi dovrei verificare l'hazard nel momento in cui \$t1 viene aggiornato nella addi e nella beq deve essere consultato in forma corretta. Sono 2 istruzioni che nel codice sono scritte a distanza di parecchie istruzioni ma con l'introduzione di jump e branch queste in realtà distano di 2 sole istruzioni (la beq viene eseguita 2 istanti dopo la addi)

Esercizio 4

lw \$t5, 8(\$t4)

add \$v0, \$v0, \$t5

L'istruzione "lw \$t5, 8(\$t4)" aggiorna il registro \$t5 nell'ultimo istante con la fase di write back, mentre la add consulta il registro durante la fase di decode ovvero al secondo istante. Anche utilizzando la soluzione "prima scrivo, poi leggo" non si riuscirebbe a risolvere l'hazard.
Soluzione: forwarding

Esercizi cache

1. In una cache direct-mapped con 32 blocchi (da 1 word), quali indirizzi di parole di memoria sono mappati nel blocco 13_{10} ?

Sono mappati nel blocco 13_{10} della cache, tutti gli indirizzi di memoria la cui rappresentazione binaria ha gli ultimi $\log_2(32) = 5$ bit uguali a 01101 (codifica binaria di 13_{10} su 5 bit)

2. L'indirizzo della parola di memoria 301_{10} fa parte degli indirizzi mappati nel blocco 13_{10} di una cache direct-mapped di 32 blocchi (da 1 word) e di una cache direct-mapped di 64 blocchi (da 1 word)?

$301_{10} \rightarrow (100101101)$

Tale indirizzo fa parte degli indirizzi mappati nel blocco 13 in una cache direct mapped a 32 blocchi (avendo gli ultimi 5 bit uguali a 01101).

NON fa invece parte degli indirizzi mappati nel blocco 13 in una cache direct mapped a 64 blocchi (non avendo gli ultimi 6 bit uguali a 001101)

3. In una cache direct mapped in cui ogni blocco contiene 2 word, qual è il valore del TAG per la parola di memoria con indirizzo 301_{10} ?

Se ad esempio sono usati per l'indice 4 bit (informazione non disponibile in questo esercizio)

$301_{10} \rightarrow (100101101)$

il valore del TAG è 1001, l'indice è 0110 e il bit meno significativo è usato per individuare la parola

4. Sia data una cache direct-mapped con blocchi di 4 parole e una dimensione totale di 32 parole. Assumendo indirizzi (di parola) a 16 bit, determinare il numero di bit riservati all'etichetta, all'indice di blocco e alla parola.

- 1) **Numero blocchi cache:** se in totale sono 32 parole, e ogni blocco contiene 4 parole, $32/4 = 8$ blocchi
- 2) **Indice:** Quindi essendo 8 blocchi in cache l'indice dell'indirizzo sarà composto da $\log_2(8) = 3$ bit
- 3) **Parola:** essendo 4 parole esse si possono individuare tramite **2 bit**
- 4) **Tag:** i bit restanti dell'indirizzo saranno associati al tag (**11 bit**)

5. Sia data una cache direct-mapped con blocchi di **4 parole** e una dimensione totale di **32 parole** e la seguente sequenza di indirizzi di parola a cui si intende accedere:

1, 4, 8, 5, 33, 66, 32, 56, 9, 11, 4, 43, 88, 6, 32

Determinare miss e hit alla cache, assumendo che la cache sia inizialmente vuota.

Gli indirizzi a cui dobbiamo accedere sono espressi come **indirizzi di parola**, trasformare in **indirizzi di blocco** dividendo per 4 (ampiezza di ogni blocco) e prendendo la parte intera del risultato della divisione.

1, 4, 8, 5, 33, 66, 32, 56, 9, 11, 4, 43, 88, 6, 32
 ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
 0, 1, 2, 1, 8, 16, 8, 14, 2, 2, 1, 10, 22, 1, 8

indirizzi di blocco: 0, 1, 2, 1, 0, 0, 0, 6, 2, 2, 1, 2, 6, 1, 0

I blocchi sono sempre 8, perché come prima $32/4 = 8$, quindi $\log_2(8) = 3$ bit

1=1 (001)

2=2 (010)

8= 0 (perché $8=1000$) 14= 6 (1110) 22= 6 (10110)

Blocco	1	2	3	4	5	6	7	8
0	mem[0]	mem[0]	mem[0]	mem[0]	mem[8]	mem[16]	mem[8]	mem[8]
1		mem[1]	mem[1]	mem[1]	mem[1]	mem[1]	mem[1]	mem[1]
2			mem[2]	mem[2]	mem[2]	mem[2]	mem[2]	mem[2]
3								
4								
5								
6								mem[14]
7								
	miss	miss	miss	hit	miss	miss	miss	miss

(non finisce così lo schema)

Nel blocco 0 memorizzo 0, nel blocco 1 memorizzo 1, nel blocco 2 memorizzo 2, nel blocco 1 memorizzo 1 ma c'è già quindi HIT, nel blocco 0 memorizzo 8 al posto dello 0 di prima, nel blocco 0 memorizzo 16 al posto dell'8 di prima, nel blocco 0 memorizzo 8, poi nel 6 memorizzo 14, nel blocco 2 c'è già 2 quindi HIT, altro HIT, HIT, miss e metto nel blocco 2 il 10, nel blocco 6 metto il 22, HIT, HIT

6. Si supponga di avere una cache a mappaggio diretto con 4 blocchi da 2 word. Se la cache parte vuota e si esegue 10 volte un loop che accede alle word che si trovano agli indirizzi 0, 1, 2, 3, 4, quanti miss ci saranno?

Al primo accesso si avrà un miss e le word che si trovano agli indirizzi 0 e 1 saranno caricate nel primo blocco della cache (quello con indice 0). Le word agli indirizzi 2 e 3 saranno caricate (dopo il secondo miss) nel blocco con indice 1. La word all'indirizzo 4 sarà caricata nel blocco con indice 2, dopo il terzo miss. Dopodiché, ripetendo il ciclo, ci saranno solo hit. In totale si avranno quindi 3 miss.

7. Considerando indirizzi di memoria a 32 bit per una cache direct-mapped con dimensione di ogni blocco di 4 parole e dimensione totale di 64 KB, qual è il numero di bit per i TAG?

$64 \text{ KB} = 64\,000 \text{ byte}$

Dimensione del blocco = 4 word, $4 \cdot (4 \text{ byte}) = 16 \text{ byte}$

$64\,000 / 16 = 4000$ numero di blocchi

$\log_2(4000) = 12$ bit indice

4 bit sono usati per individuare uno dei 4 byte all'interno di una delle 4 parole

- blocchi da 4 parole a 2 bit per individuare la parola
- indirizzo al byte a 2 bit per individuare il byte all'interno della parola

Se l'indirizzo era di 32 bit, 16 bit sono per etichetta

8. Quanti bit sono necessari per una cache a mappaggio diretto con 16 KB di dati e blocchi da 4 word, assumendo un indirizzamento a 32 bit?

$16000 / 4 \cdot 4 = 1000$ blocchi

$\log_2(1000) = 10$ bit indice

Bit tag: (4 bit riguardano l'individuazione della parola, 2 bit per la parole, 2 bit per individuare il byte della parola) quindi $32 - 14 = 18$ bit tag

Bit tot: $1000 \cdot (4 \cdot 32 \text{ bit dati}, 18 \text{ tag}, 1 \text{ bit validità})$

$1000 \cdot (128 + 18 + 1) = 147\,000 \text{ bit} = 147 \text{ Kbit}$

9. Data una macchina che richiede 5 cicli per accessi solo in cache (hit), 15 cicli di miss penalty e hit probability 0,75, qual è il numero medio di cicli per accesso alla memoria? CPI indica il numero di cicli di clock necessari al microprocessore per eseguire un'istruzione.

CPI = 5 cicli per istruzione quando non ci sono miss

miss penalty = 15 cicli

hit prob = 0,75

miss prob = 0,25

Num medio di cicli per accesso in memoria:

$$0,75 \cdot 5 + 0,25 \cdot 15 = 3,75 + 3,75 = 7,5$$

10. Data una macchina con frequenza di clock di 600 MHz, 2 CPI per accessi solo in cache quando non ci sono miss, 22 cicli di miss penalty e hit probability 0,95, qual è velocità media in istruzioni/sec?

Velocità media (istr/sec) sarà pari alla frequenza (cicli/sec) divisa per il numero medio dei cicli eseguiti per ogni istruzione, ovvero:

$$\frac{600 \text{ MHz}}{(0,95 \cdot 2 + 0,05 \cdot 22)} = \frac{600 \text{ milioni}}{(1,9 + 1,1)} = 200 \text{ milioni istr/sec}$$

11. Si considerino due cache, C1 e C2, costituite entrambe da 4 blocchi di due word. La cache C1 è direct-mapped, la cache C2 è fullyassociative con politica di rimpiazzamento LRU

Determinare, nei due casi, il numero di miss per la seguente sequenza di operazioni di lettura di una word agli indirizzi:

C1: 508, 1016, 510, 24, 540, 1050, 1020, 24, 538

- Gli indirizzi sono al byte, espressi in decimale
- Si assuma che all'inizio le cache siano vuote
- Spiegare il ragionamento che ha portato alla risposta

$$508/8 = 63 \quad 63\%4 = 3 \text{ blocco} \quad \text{miss}$$

$$1016/8 = 127 \quad 127\%4 = 3 \text{ blocco} \quad \text{miss}$$

$$011111 = 3$$

Si sarebbe potuto concludere che si hanno solo miss, osservando che tutti gli indirizzi sono mappati all'indice 3 e che non ci sono due accessi consecutivi che richiedono lo stesso indirizzo.

C2: Essendo i blocchi da 2 word e gli indirizzi al byte, dividiamo tutti gli indirizzi per 8 (e consideriamo la parte intera del risultato), dopodichè verifichiamo se abbiamo miss o hit e nel caso di miss applichiamo la politica LRU

$508/8 = 63$ miss (63 -> 0)

$1016/8 = 127$ miss (127 -> 1)

$510/8 = 63$ hit -> 0

$24/8 = 3$ miss (3 -> 2)

$540/8 = 67$ miss (67 -> 3)

$1050/8 = 131$ miss (131 -> 1) il blocco 1 è quello che non si utilizza da più tempo

$1020/8 = 127$ miss (127 -> 0) LRU

$24/8 = 3$ hit

$538/8 = 67$ hit